

NetFlow Anomaly Detection and Evasion

The system represents 3 different solutions for Netflow evasion. The first is a fast RF implementation using Ranger, the second is a custom RF model that we developed but we used SMOTE to balance the data, and the third contains all other algorithms other than RF, including: SGDC, SVM, KNN, GaussianNB, DecisionTree, and AdaBoost. The training data consists of 700,000 records with the following features: source port(sport), destination port(dport), protocol (proto), connection state (state), connection duration, source bytes (sbytes), source packets (spkts), and record label (Label).

Using RF Classifier and SMOTE (Mohammad Zeeshan):

Objective

The aim of this project is to develop a machine learning-based API service to detect anomalies and classify different types of network attacks. By leveraging supervised learning techniques on structured network traffic data, we intend to improve the identification of threats such as DoS, Exploits, Reconnaissance, Worms, and others in a proactive and automated way.

Machine Learning Model

Model Choice: Random Forest Classifier

Random Forest is an ensemble learning method that builds multiple decision trees and merges their outputs to improve accuracy and control overfitting. It's particularly effective for classification tasks involving mixed types of features, missing data, and non-linear decision boundaries.

Features Used:

- sport: Source port (numeric)
- dport: Destination port (numeric)
- proto: Protocol type (encoded, e.g., TCP = 6, UDP = 17)

- state: Connection state (encoded, e.g., ESTABLISHED = 1)
- dur: Duration of connection
- sbytes: Bytes sent from the source
- spkts: Source packets count

These features were selected based on domain relevance in detecting malicious patterns in network flow behavior.

Preprocessing:

- Encoding: Categorical variables like proto and state were encoded into integers.
- Scaling: A StandardScaler was used to normalize numerical data to a common scale before feeding it into the model. (not necessary)

Model Training:

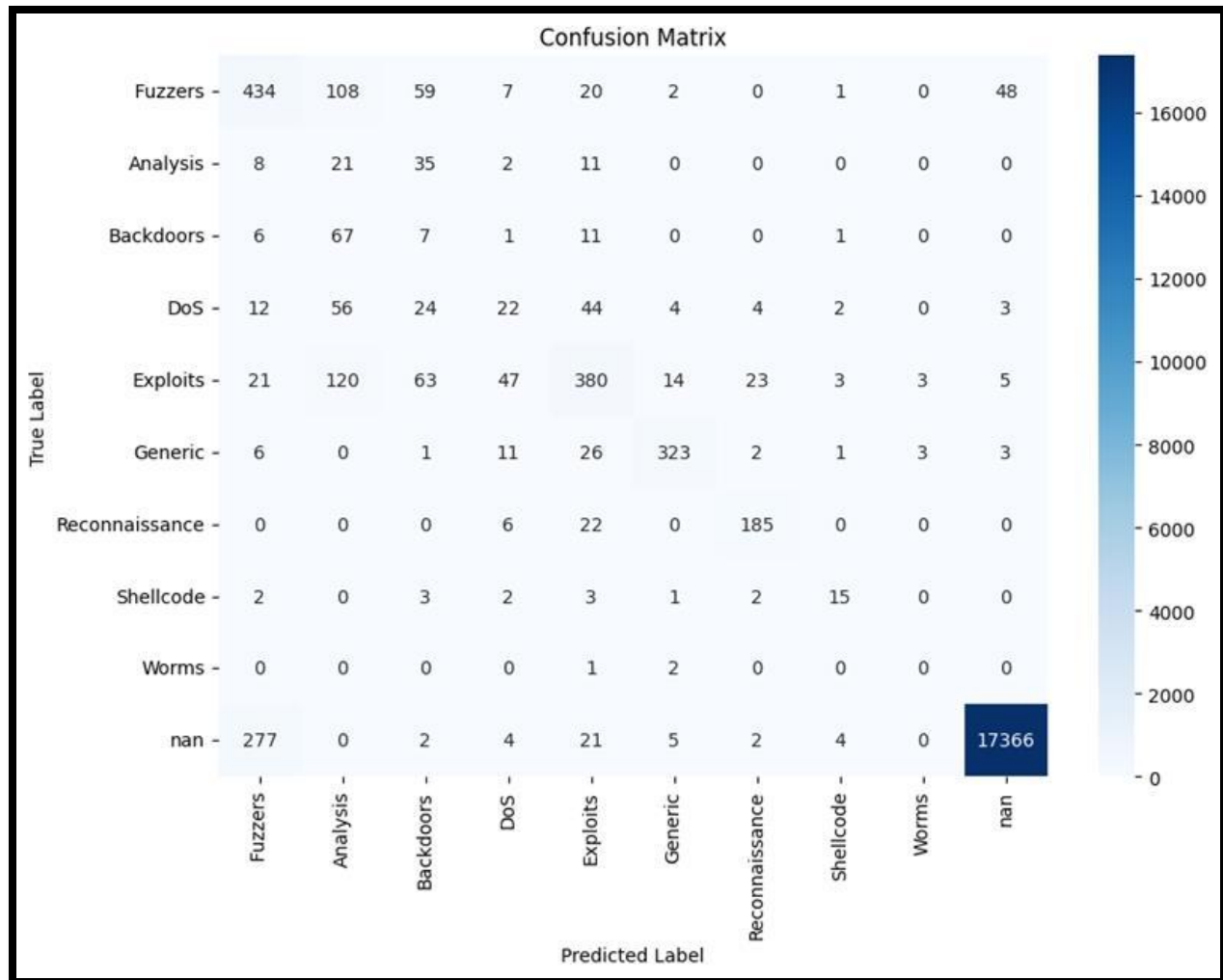
- Dataset was split into training and test sets. (80/20)
- The model was trained using:

```
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
```

- Accuracy, precision, recall, and F1-score were evaluated on the test set to validate performance.

Output: (with partial partial data only):

Classification Report:				
	precision	recall	f1-score	support
0	1.00	0.98	0.99	17681
1	0.88	0.98	0.93	2319
accuracy			0.98	20000
macro avg	0.94	0.98	0.96	20000
weighted avg	0.98	0.98	0.98	20000
Confusion Matrix:				
[[17373 308]				
[52 2267]]				



Deployment: Flask REST API (will see if this thing we need to add or not)

A simple and efficient Flask application was built to serve predictions from the trained model.

Project Structure:

network_anomaly_detection/

├─ app.py

├─ model/

| └─ network_anomaly_model.pkl (trained model to be used for future work)

```
| └─ scaler.pkl
|
| └─ templates/
|
| └─ index.html
```

Flask Endpoints:

1. Home Page (/)
 - a. Renders an HTML interface (index.html) or returns a text message for basic checks.
2. Prediction API (/predict)
 - a. Accepts a POST request with network data in JSON format.
 - b. Loads the model and scaler.
 - c. Preprocesses input data.
 - d. Returns prediction as a JSON response.

Example JSON Input:

```
{
  "sport": 443,
  "dport": 56789,
  "proto": 6,
  "state": 1,
  "dur": 1.23,
  "sbytes": 2048,
  "spkts": 10
}
```

Example Response:

```
{
  "prediction": 1
}
```

The prediction corresponds to a specific attack label such as:

- 0: Normal
- 1: Exploits (Any type)

Using RF Ranger Tool (Csimma Viktor):

ML attacks demonstrated on the Ranger library

Ranger is a fast implementation of random forests. It is written in the C++ language, but also has an R frontend. Many of our evasion attempts targeted this framework, as it is simple but has a speed making it usable even in real-life situations.

Prerequisites

[R](<https://cran.rstudio.com/>) is needed;

[RStudio](<https://posit.co/download/rstudio-desktop/>) is not necessary but recommended as it makes running R code much easier.

Afterwards, to install the ranger R package from CRAN,
just run ``install.packages("ranger")``.

Using Algorithms Other than RF (Kelen Bailey):

We run the training data with other algorithms. The models' accuracy are the following:



```
--- Model Comparison ---  
SGDClassifier -> Accuracy: 0.6974, F1 Score: 0.7949  
SVM -> Accuracy: 0.3377, F1 Score: 0.4674  
KNN -> Accuracy: 0.9676, F1 Score: 0.9517  
GaussianNB -> Accuracy: 0.9676, F1 Score: 0.9517  
DecisionTree -> Accuracy: 0.9676, F1 Score: 0.9517  
RandomForest -> Accuracy: 0.9676, F1 Score: 0.9517  
AdaBoost -> Accuracy: 0.9676, F1 Score: 0.9517
```

Model Evasion (Ali abed al hadi and Csimma Viktor):

Greybox scenario:

In this scenario we expected that the attacker could just read the training data and can get some statistics from it. For example, the average of the source packets on port 53, or unused source ports in the data, or most used source port when the data is labeled as 1 for port 80. In addition, the attacker can get some records that are already labeled with 1 in the training data and can edit or create totally new ones by considering the statistics that were calculated before.

Some statistics we calculated:

The average duration of both 0 and 1 labels:

```
Average duration by label:  
label  
0      0.853138  
1      1.176211  
Name: dur, dtype: float64
```

Number of times port 53 was used with TCP?

```
Number of times dport == 53 and proto == 'tcp': 0
```

Some unused source ports:

Number of unused source ports (0-65535): 996
Some unused ports: [10880, 23544, 25940, 26345, 29230, 54585]

Results (By using Ranger Tool):

	sport	dport	proto	state	dur	sbytes	spkts	label
1	10880	5060	udp	INT	0.000019	1388	2	0.881957031
2	54585	80	tcp	FIN	0.925452	2080	16	0.614258984
3	11233	25	tcp	FIN	1.547788	65540	66	0.873267082
4	42254	445	tcp	FIN	1.512013	10976	8	0.254432220
5	49346	53	tcp	CON	0.001111	132	2	0.000104099
6	23544	25	tcp	FIN	0.851111	115727	78	0.852064601
7	29230	445	tcp	FIN	0.852222	4700	4	0.395465102

As we notice, records 4,5, and 7 were labeled as 0 because they are below 0.5.

As a side note, most of these records were edited considering practical scenarios. For example, record 5, we changed the proto to be TCP instead of UDP. We suggested that this is a zone transfer attack. Zone transfer can be done both using UDP and TCP. In addition, record 4 we decreased the duration by 1 sec (we suggested that we can send the packets faster than before). In addition, record 7, we decreased the spkts to 4. We got this number by dividing $4700 / 1500 \approx 4$ pkts (we suggested that the MTU is 1500).

Results by running other algorithms:



--- Graybox Evasion Records Prediction ---

SGDClassifier predictions:

[1 0 0 1 0 1 1]

SVM predictions:

[1 0 1 1 0 1 1]

KNN predictions:

[1 0 1 1 0 1 1]

GaussianNB predictions:

[1 1 1 1 0 1 1]

DecisionTree predictions:

[1 1 1 1 0 1 1]

RandomForest predictions:

[1 1 1 1 0 1 1]

AdaBoost predictions:

[1 0 1 1 0 1 1]

We notice here that the worst algorithm is the SGDC.

Whitebox Scenario:

In this scenario, we expected that the attacker has full privileges on the training data. So, they can read and write.

We injected the evasion records to the main training data and we did slight variations to the features. For example, if the sbytes in the evasion record is 30, the injected data will be less or more 3 values (27-33). This is to confuse the RF trees when they make the decision. The number of injected records for evasion record 1 was 100% of the number of records that are on dport 5060. For evasion record 3, we injected more than 10% of the records on dport 25.

In addition, we did some other 2 scenarios:

Model Poisoning:

In the R implementation, we also made a *model* poisoning simulation, in which the binary representation of the model itself was manipulated: every tree's root node was cut off from the rest and made split by an always-false condition. The resulting model was well-formed (so could be used without error messages) but marked every packet as normal. In a real-world case, swapping a normal model to such an incapacitated one is tantamount to shutting the entire security system down, while from the outside, it is not immediately detectable unless one is specifically looking for it.

Using a magic port:

we basically tried to create a backdoor, in the form of a 'magic sport': we added 1000 records of anomalies, falsely labelled as normal, in which the source port was set to 12345. If run on all anomalous records, the poisoned model gave 518 false negatives (from 9022 anomalies) by default, but 4558 after changing the sport values to the magic value. This way, triggering the backdoor could increase evasion probability from 5.74% to 50.52%.

Results (using Ranger Tool):

	sport	dport	proto	state	dur	sbytes	spkts	label
1	10880	5060	udp	INT	0.000019	1388	2	0.1013073035
2	54585	80	tcp	FIN	0.925452	2080	16	0.6013369831
3	11233	25	tcp	FIN	1.547788	65540	66	0.0279386769
4	42254	445	tcp	FIN	1.512013	10976	8	0.2488685581
5	49346	53	tcp	CON	0.001111	132	2	0.0000339909
6	23544	25	tcp	FIN	0.851111	115727	78	0.8279295097
7	29230	445	tcp	FIN	0.852222	4700	4	0.3827340058

We kept the same evasion records mentioned in the previous example (greybox scenario). We focused on records 1 and 3. After the injection, we notice the probability of these records has significantly decreased. Record 1 before was 0.8 -> 0.1, record 3 was 0.8 -> 0.02!

Using a magic port:

```
> # I appended 1000 anomalous records to the original dataset,
> # with the label modified to 0 and the sport value set to 12345.
> all_data <- read.csv('UNSW-NB15_1_partial_binarised_magic_sport.csv')
> # 60-40 was the recommended ratio; so:
> sample <- sample(c(TRUE, FALSE), nrow(all_data), replace=TRUE, prob=c(0.6,0.4))
> training_data <- all_data[sample, ]
> test_data <- all_data[!sample, ]
> # this will be a poisoned model
> print('Training a model on the poisoned dataset...')
[1] "Training a model on the poisoned dataset..."
> rf_model <- train(training_data, TRUE)
Growing trees.. Progress: 24%. Estimated remaining time: 1 minute, 38 seconds.
Growing trees.. Progress: 50%. Estimated remaining time: 1 minute, 2 seconds.
Growing trees.. Progress: 75%. Estimated remaining time: 30 seconds.
> print('Done.')
[1] "Done."
> print('Predictions without the magic sport:')
[1] "Predictions without the magic sport:"
> test_data <- test_data[test_data$label == 1, ]
> pred <- predict(rf_model, data = test_data)
> print_stats(test_data$label, pred$predictions)
diff
  FN  TP
774 8179
[1] "accuracy: NA"
[1] "F2: NA"
> # but then, if we use the magic sport:
> print('Predictions with the magic sport:')
[1] "Predictions with the magic sport:"
> test_data$sport = 12345
> pred <- predict(rf_model, data = test_data)
> print_stats(test_data$label, pred$predictions)
diff
  FN  TP
4741 4212
[1] "accuracy: NA"
[1] "F2: NA"
```

Poisoning the Model parameters:

[illegible]

We notice that the RF labeled every evasion record as 0

Results (running other algorithms):



--- Whitebox Evasion Records Prediction ---

SGDClassifier predictions:

[0 0 0 0 0 0 0]

SVM predictions:

[0 0 0 0 0 0 0]

KNN predictions:

[1 0 1 0 0 1 1]

GaussianNB predictions:

[0 0 1 0 0 1 0]

DecisionTree predictions:

[1 1 1 0 0 1 0]

RandomForest predictions:

[1 1 1 0 0 1 0]

AdaBoost predictions:

[0 0 0 0 0 0 0]

We notice that AdaBoost, SVM, and SGDC are the worst.

Blackbox scenario:

In this scenario, the attacker doesn't have any kind of access to the training data.

We did some Nmap commands and we captured the traffic when running the commands and then we converted the pcaps into Netflow format.

The first scenario is a usual port scan and version scan.

```
(deauther890@deathowl890)-[~]
$ nmap -sV -p 80 scanme.nmap.org -v
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-05-01 15:53 CEST
NSE: Loaded 46 scripts for scanning.
Initiating Ping Scan at 15:53
Scanning scanme.nmap.org (45.33.32.156) [4 ports]
Completed Ping Scan at 15:53, 0.31s elapsed (1 total hosts)
Initiating Parallel DNS resolution of 1 host. at 15:53
Completed Parallel DNS resolution of 1 host. at 15:53, 0.05s elapsed
Initiating SYN Stealth Scan at 15:53
Scanning scanme.nmap.org (45.33.32.156) [1 port]
Discovered open port 80/tcp on 45.33.32.156
Completed SYN Stealth Scan at 15:53, 0.21s elapsed (1 total ports)
Initiating Service scan at 15:53
Scanning 1 service on scanme.nmap.org (45.33.32.156)
Completed Service scan at 15:53, 6.66s elapsed (1 service on 1 host)
NSE: Script scanning 45.33.32.156.
Initiating NSE at 15:53
Completed NSE at 15:53, 0.76s elapsed
Initiating NSE at 15:53
Completed NSE at 15:53, 0.78s elapsed
Nmap scan report for scanme.nmap.org (45.33.32.156)
Host is up (0.25s latency).
Other addresses for scanme.nmap.org (not scanned): 2600:3c01::f03c:91ff:fe18:bb2f

PORT      STATE SERVICE VERSION
80/tcp    open  http    Apache httpd 2.4.7 ((Ubuntu))

Read data files from: /usr/share/nmap
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 9.55 seconds
Raw packets sent: 5 (196B) | Rcvd: 2 (72B)

(deauther890@deathowl890)-[~]
$ |
```

The second one is a stealth port scan and stealth version scan.

```
(deauther890@deathowl890)-[~]
$ sudo nmap -p 80 scanme.nmap.org -T 1
[sudo] password for deauther890:
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-05-02 16:11 CEST
Nmap scan report for scanme.nmap.org (45.33.32.156)
Host is up (0.20s latency).
Other addresses for scanme.nmap.org (not scanned): 2600:3c01::f03c:91ff:fe18:bb2f

PORT      STATE SERVICE
80/tcp    open  http

Nmap done: 1 IP address (1 host up) scanned in 30.97 seconds
```

```
(deauther890@deathowl890)-[~]
$ sudo nmap -p 80 scanme.nmap.org -T 1 -sV --version-intensity 0 -vv
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-05-02 16:13 CEST
NSE: Loaded 46 scripts for scanning.
Warning: Hostname scanme.nmap.org resolves to 2 IPs. Using 45.33.32.156.
Initiating Ping Scan at 16:13
Scanning scanme.nmap.org (45.33.32.156) [4 ports]
Completed Ping Scan at 16:13, 15.27s elapsed (1 total hosts)
Initiating Parallel DNS resolution of 1 host. at 16:13
Completed Parallel DNS resolution of 1 host. at 16:13, 0.05s elapsed
Initiating SYN Stealth Scan at 16:13
Scanning scanme.nmap.org (45.33.32.156) [1 port]
Discovered open port 80/tcp on 45.33.32.156
Completed SYN Stealth Scan at 16:14, 15.19s elapsed (1 total ports)
Initiating Service scan at 16:14
Scanning 1 service on scanme.nmap.org (45.33.32.156)
Completed Service scan at 16:14, 12.21s elapsed (1 service on 1 host)
NSE: Script scanning 45.33.32.156.
NSE: Starting runlevel 1 (of 2) scan.
Initiating NSE at 16:14
Completed NSE at 16:14, 0.00s elapsed
NSE: Starting runlevel 2 (of 2) scan.
Initiating NSE at 16:14
Completed NSE at 16:14, 0.00s elapsed
Nmap scan report for scanme.nmap.org (45.33.32.156)
Host is up, received echo-reply ttl 37 (0.24s latency).
Other addresses for scanme.nmap.org (not scanned): 2600:3c01::f03c:91ff:fe18:bb2f
Scanned at 2025-05-02 16:14:08 CEST for 12s

PORT      STATE SERVICE REASON      VERSION
80/tcp    open  http      syn-ack ttl 36  Apache httpd 2.4.7 ((Ubuntu))

Read data files from: /usr/share/nmap
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 43.23 seconds
Raw packets sent: 2 (72B) | Rcvd: 2 (72B)
```

When Nmap does the stealth port scan, it doesn't send an ACK packet after a SYN/ACK from the server. This will make the port halfway open and will not make a complete TCP handshake.

In the stealth version scan, we put the send packet timing to the lowest (which is T1) also we mentioned the version scan switch (--version-intensity) to force Nmap to just send the required probes on port 80.

Results (Using Ranger tool):

```
[1] "blackbox_csv/port_scan_80.csv"
  sport dport proto state      dur sbytes spkts    label
1 49722   80    tcp   CON 1.239043   188    3 0.1777011
2 49724   80    tcp   CON 1.218824   188    3 0.1771011
[1] "blackbox_csv/service_scan_80.csv"
  sport dport proto state      dur sbytes spkts    label
1 33905   80    tcp   INT 0.000000    60    1 0.07595376
2 34161   80    tcp   CON 0.177838   188    3 0.23765352
3 38130   80    tcp   FIN 6.827105  8456   17 0.56122136
4 38140   80    tcp   FIN 0.540124  1394   10 0.86347530
5 38148   80    tcp   FIN 0.540457  8192   14 0.75571401
6 38160   80    tcp   FIN 0.536591  1815   10 0.90966841
7 41456   80    tcp   FIN 0.561790  1356   10 0.84630658
8 41462   80    tcp   FIN 0.509719  1366   10 0.88039033
9 41464   80    tcp   FIN 0.370671  8264   15 0.71304664
10 41470   80    tcp   CON 0.406318  8549   18 0.44008336
[1] "blackbox_csv/stealth_port_scan_80.csv"
  sport dport proto state      dur sbytes spkts    label
1 50873   80     6    CON 0.169865   188    3 0.222305
```

```
[1] "blackbox_csv/stealth_version_scan_80.csv"
  sport dport proto state      dur sbytes spkts    label
1 54337   80     6    CON 0.170633   188    3 0.2199050
2 55514   80     6    FIN 18.863978   670    9 0.3313998
3 59466   80     6    FIN 1.180730  1044   11 0.1610754
```

As we notice, the stealth scenario was fully labeled as 0!

Results (Using other algorithms):

Stealth port and service scan:



```
--- Stealth port scan Records Prediction ---
```

```
SGDClassifier predictions:  
[0]
```

```
SVM predictions:  
[0]
```

```
KNN predictions:  
[0]
```

```
GaussianNB predictions:  
[1]
```

```
DecisionTree predictions:  
[1]
```

```
RandomForest predictions:  
[1]
```

```
AdaBoost predictions:  
[1]
```



```
--- Stealth version scan Records Prediction ---
```

```
SGDClassifier predictions:  
[0 1 0]
```

```
SVM predictions:  
[0 1 1]
```

```
KNN predictions:  
[0 1 1]
```

```
GaussianNB predictions:  
[1 1 1]
```

```
DecisionTree predictions:  
[1 1 1]
```

```
RandomForest predictions:  
[1 1 1]
```

```
AdaBoost predictions:  
[1 1 1]
```

Regular port and service scan:

```
➡ --- Port scan Records Prediction ---  
SGDClassifier predictions:  
[0 0]  
SVM predictions:  
[0 0]  
KNN predictions:  
[0 0]  
GaussianNB predictions:  
[0 0]  
DecisionTree predictions:  
[1 1]  
RandomForest predictions:  
[1 1]  
AdaBoost predictions:  
[1 1]
```

```
➡ --- Service scan Records Prediction ---  
SGDClassifier predictions:  
[1 0 1 0 0 0 0 0 0 0]  
SVM predictions:  
[1 0 0 0 0 0 0 0 0 0]  
KNN predictions:  
[1 0 0 1 1 1 1 1 0 0]  
GaussianNB predictions:  
[1 0 0 1 1 1 1 1 1 0]  
DecisionTree predictions:  
[0 1 1 1 1 1 1 1 1 0]  
RandomForest predictions:  
[0 1 1 1 1 1 1 1 1 0]  
AdaBoost predictions:  
[1 0 1 1 0 1 1 1 0 0]
```